Part 2: Unconstrained Optimization

2.1 Single-Variable Unconstrained Optimization

Find x* that maximizes f(x) where f(x) is continuously differentiable.

In [ ]:
```
# Example 2.1: Single-variable optimization (from slides)
# Problem: max f(x) = 1000000 + 4000x - x²

def profit_function(x):
    """Profit function: f(x) = 1000000 + 4000x - x²"""
    return 1000000 + 4000*x - x**2

def profit_derivative(x):
    """Derivative: f'(x) = 4000 - 2x"""
    return 4000 - 2*x

def profit_second_derivative(x):
    """Second derivative: f''(x) = -2"""
    return -2

# Analytical solution
x_optimal_analytical = 2000
f_optimal_analytical = profit_function(x_optimal_analytical)

print("="*60)
print("Example 2.1: Single-Variable Optimization")
print("="*60)
print(f"Problem: max f(x) = 1000000 + 4000x - x²")
print()
print("Analytical Solution:")
print(f"First-order condition: f'(x*) = 4000 - 2x = 0")
print(f"Optimal point: x* = {x_optimal_analytical}")
print(f"Second derivative: f''(x) = {profit_second_derivative(x_optimal_analytical)
print(f"Optimal value: f(x*) = {f_optimal_analytical:,.0f}")
print()

# Numerical solution using scipy
result = minimize_scalar(lambda x: -profit_function(x), bounds=(0, 5000), method='b
x_optimal_numerical = result.x
f_optimal_numerical = -result.fun

print("Numerical Solution (SciPy):")
print(f"Optimal point: x* = {x_optimal_numerical:.6f}")
print(f"Optimal value: f(x*) = {f_optimal_numerical:,.2f}")
print(f"Error: {abs(x_optimal_analytical - x_optimal_numerical):.2e}")
print()
```

2.2 Multi-Variable Unconstrained Optimization

Find **x*** that maximizes $f(x_1, x_2, ..., x_n)$

In [ ]:
```
# Example 2.2: Multi-variable optimization (from slides)
# Problem: max f(x₁, x₂) = 1000000 + 300x₁ + 500x₂ - x₁² - x₂²

def profit_2d(x):
    """2D profit function: f(x₁, x₂) = 1000000 + 300x₁ + 500x₂ - x₁² - x₂²"""
    return 1000000 + 300*x[0] + 500*x[1] - x[0]**2 - x[1]**2

def profit_2d_gradient(x):
    """Gradient: ∇f = [300 - 2x₁, 500 - 2x₂]"""
    return np.array([300 - 2*x[0], 500 - 2*x[1]])

def profit_2d_hessian(x):
    """Hessian matrix: H = [[-2, 0], [0, -2]]"""
    return np.array([[-2, 0], [0, -2]])

# Analytical solution
# From first-order conditions: 300 - 2x₁ = 0 and 500 - 2x₂ = 0
x_optimal_analytical = np.array([150, 250])
f_optimal_analytical = profit_2d(x_optimal_analytical)

print("="*60)
print("Example 2.2: Multi-Variable Optimization")
print("="*60)
print(f"Problem: max f(x₁, x₂) = 1000000 + 300x₁ + 500x₂ - x₁² - x₂²")
print()
print("Analytical Solution:")
print(f"First-order conditions:")
print(f"  ∂f/∂x₁ = 300 - 2x₁ = 0  =>  x₁* = 150")
print(f"  ∂f/∂x₂ = 500 - 2x₂ = 0  =>  x₂* = 250")
print()
print(f"Optimal point: x* = {x_optimal_analytical}")
print(f"Optimal value: f(x*) = {f_optimal_analytical:,.0f}")
print()
print("Second-order condition (Hessian):")
H = profit_2d_hessian(x_optimal_analytical)
eigenvalues = np.linalg.eigvals(H)
print(f"H = \n{H}")
print(f"Eigenvalues: {eigenvalues}")
print(f"Negative definite (all eigenvalues < 0): {all(eigenvalues < 0)} ✓ (Maximum)
print()

# Numerical solution
result = minimize(lambda x: -profit_2d(x), x0=np.array([0, 0]), method='BFGS')
x_optimal_numerical = result.x
f_optimal_numerical = -result.fun

print("Numerical Solution (SciPy - BFGS):")
print(f"Optimal point: x* = {x_optimal_numerical}")
print(f"Optimal value: f(x*) = {f_optimal_numerical:,.2f}")
print(f"Error in x₁: {abs(x_optimal_analytical[0] - x_optimal_numerical[0]):.2e}")
print(f"Error in x₂: {abs(x_optimal_analytical[1] - x_optimal_numerical[1]):.2e}")
print()
```

2.3 Additional Examples: Rosenbrock and Sphere Functions

In [ ]:
```
# Example 2.3: Additional unconstrained optimization examples

def sphere_function(x):
    """Sphere function: f(x) = Σ xᵢ²"""
    return np.sum(x**2)

def rosenbrock_function(x):
    """Rosenbrock function: f(x) = Σ [100(x_{i+1} - xᵢ²)² + (1 - xᵢ)²]"""
    return sum(100.0*(x[1:]-x[:-1]**2.0)**2.0 + (1-x[:-1])**2.0)

def rastrigin_function(x):
    """Rastrigin function: f(x) = 10n + Σ [xᵢ² - 10cos(2πxᵢ)]"""
    return 10*len(x) + sum(x**2 - 10*np.cos(2*np.pi*x))

print("="*60)
print("Example 2.3: Optimization of Standard Test Functions")
print("="*60)
print()

# Optimize Sphere function
print("1. SPHERE FUNCTION: f(x) = Σ xᵢ²")
print("-" * 40)
x0_sphere = np.array([5.0, 5.0, 5.0])
result_sphere = minimize(sphere_function, x0_sphere, method='BFGS')
print(f"Initial point: {x0_sphere}")
print(f"Optimal point: {result_sphere.x}")
print(f"Optimal value: {result_sphere.fun:.2e}")
print(f"Success: {result_sphere.success}")
print(f"Iterations: {result_sphere.nit}")
print()

# Optimize Rosenbrock function
print("2. ROSENBROCK FUNCTION: f(x) = Σ [100(x_{i+1} - xᵢ²)² + (1 - xᵢ)²]")
print("-" * 40)
x0_rosenbrock = np.array([-1.2, 1.0])
result_rosenbrock = minimize(rosenbrock_function, x0_rosenbrock, method='BFGS')
print(f"Initial point: {x0_rosenbrock}")
print(f"Optimal point: {result_rosenbrock.x}")
print(f"Optimal value: {result_rosenbrock.fun:.2e}")
print(f"Success: {result_rosenbrock.success}")
print(f"Iterations: {result_rosenbrock.nit}")
print()

# Optimize Rastrigin function (multimodal)
print("3. RASTRIGIN FUNCTION: f(x) = 10n + Σ [xᵢ² - 10cos(2πxᵢ)]")
print("-" * 40)
x0_rastrigin = np.array([2.0, 2.0])
result_rastrigin = minimize(rastrigin_function, x0_rastrigin, method='L-BFGS-B')
print(f"Initial point: {x0_rastrigin}")
print(f"Optimal point: {result_rastrigin.x}")
print(f"Optimal value: {result_rastrigin.fun:.2e}")
print(f"Success: {result_rastrigin.success}")
print(f"Iterations: {result_rastrigin.nit}")
```

```
print(f"Note: Rastrigin is highly multimodal; local optimum found.")
print()
```

Part 3: Linear Programming (LP)

3.1 Linear Programming Formulation

max c^T x subject to **Ax ≤ b** and **A_eq x = b_eq**

In [ ]:
```
# Example 3.1: Linear Programming
print("="*60)
print("Example 3.1: Linear Programming")
print("="*60)
print()

# Problem: A company produces two products
# Maximize profit: 3x₁ + 2x₂
# Subject to:
# 2x₁ + x₂ ≤ 100  (Resource 1)
# x₁ + x₂ ≤ 80    (Resource 2)
# x₁, x₂ ≥ 0

print("Problem: Production Optimization")
print("-" * 40)
print("Maximize: 3x₁ + 2x₂ (profit)")
print("Subject to:")
print("  2x₁ + x₂ ≤ 100  (Resource 1)")
print("  x₁ + x₂ ≤ 80    (Resource 2)")
print("  x₁, x₂ ≥ 0")
print()

# Coefficients
c = np.array([3, 2])  # Objective coefficients (we'll minimize -c for maximization)
A_ub = np.array([[2, 1], [1, 1]])  # Inequality constraint matrix
b_ub = np.array([100, 80])  # Inequality constraint bounds
x_bounds = [(0, None), (0, None)]  # Non-negativity constraints

# Solve using linprog (minimizes c^T x, so we negate c)
result_lp = linprog(-c, A_ub=A_ub, b_ub=b_ub, bounds=x_bounds, method='highs')

print("Solution (SciPy - linprog):")
print(f"Optimal point: x₁ = {result_lp.x[0]:.2f}, x₂ = {result_lp.x[1]:.2f}")
print(f"Optimal value: {-result_lp.fun:.2f}")
print()

# Verify constraints
print("Constraint verification:")
print(f"  2x₁ + x₂ = {2*result_lp.x[0] + result_lp.x[1]:.2f} ≤ 100 ✓")
print(f"  x₁ + x₂ = {result_lp.x[0] + result_lp.x[1]:.2f} ≤ 80 ✓")
print(f"  x₁ = {result_lp.x[0]:.2f} ≥ 0 ✓")
print(f"  x₂ = {result_lp.x[1]:.2f} ≥ 0 ✓")
print()
```

Part 4: Quadratic Programming (QP)

4.1 Quadratic Programming Formulation

max (1/2) x^T Q x + c^T x subject to **Ax ≤ b**

In [ ]:
```
# Example 4.1: Quadratic Programming
print("="*60)
print("Example 4.1: Quadratic Programming")
print("="*60)
print()

# Problem: Portfolio optimization
# Minimize: (1/2) x^T Q x - r^T x  (minimize risk - expected return)
# Subject to: Σ xᵢ = 1 (budget constraint), xᵢ ≥ 0

# Covariance matrix (risk)
Q = np.array([[4, 1, 0.5],
              [1, 3, 0.2],
              [0.5, 0.2, 2]])

# Expected returns
r = np.array([0.05, 0.08, 0.06])

# Objective: minimize (1/2) x^T Q x - r^T x
def qp_objective(x):
    return 0.5 * x @ Q @ x - r @ x

def qp_gradient(x):
    return Q @ x - r

# Constraints: Σ xᵢ = 1, xᵢ ≥ 0
constraints = ({'type': 'eq', 'fun': lambda x: np.sum(x) - 1})
bounds = [(0, 1) for _ in range(3)]

# Initial guess
x0 = np.array([0.33, 0.33, 0.34])

# Solve QP
result_qp = minimize(qp_objective, x0, method='SLSQP', bounds=bounds, constraints=c

print("Problem: Portfolio Optimization")
print("-" * 40)
print("Minimize: (1/2) x^T Q x - r^T x  (Risk - Return)")
print("Subject to:")
print("  Σ xᵢ = 1 (Budget constraint)")
print("  xᵢ ≥ 0 (Non-negativity)")
print()
print(f"Covariance Matrix Q:\n{Q}")
print()
print(f"Expected Returns r: {r}")
```

```
print()
print("Solution:")
print(f"Optimal allocation: x = {result_qp.x}")
print(f"Allocation percentages: {result_qp.x * 100}%")
print(f"Objective value: {result_qp.fun:.6f}")
print(f"Expected return: {r @ result_qp.x:.6f}")
print(f"Portfolio variance: {result_qp.x @ Q @ result_qp.x:.6f}")
print(f"Portfolio std dev: {np.sqrt(result_qp.x @ Q @ result_qp.x):.6f}")
print()

# Compare with equal weighting
x_equal = np.array([1/3, 1/3, 1/3])
print("Comparison with equal weighting:")
print(f"Equal weights: {x_equal}")
print(f"Expected return: {r @ x_equal:.6f}")
print(f"Portfolio variance: {x_equal @ Q @ x_equal:.6f}")
print(f"Portfolio std dev: {np.sqrt(x_equal @ Q @ x_equal):.6f}")
```

Part 5: Nonlinear Programming (NLP)

5.1 Constrained Nonlinear Optimization

In [ ]:

```
Example 5.1: Constrained Nonlinear Optimization
print("="*60)
print("Example 5.1: Constrained Nonlinear Optimization")
print("="*60)
print()

# Problem from slides: min x₁ + x₂ subject to x₁² + x₂² - 2 = 0
def nlp_objective(x):
    """Objective: f(x) = x₁ + x₂"""
    return x[0] + x[1]

def nlp_constraint(x):
    """Constraint: x₁² + x₂² - 2 = 0"""
    return x[0]**2 + x[1]**2 - 2

print("Problem: Minimize x₁ + x₂")
print("Subject to: x₁² + x₂² - 2 = 0")
print()

# Constraints
constraints = {'type': 'eq', 'fun': nlp_constraint}

# Try different initial points
initial_points = [np.array([1.0, 1.0]), np.array([0.5, 1.5]), np.array([-1.0, -1.0]

print("Solutions from different starting points:")
print("-" * 40)
solutions = []
for i, x0 in enumerate(initial_points):
    result = minimize(nlp_objective, x0, method='SLSQP', constraints=constraints)
```

```
    solutions.append(result)
    print(f"\nStart point {i+1}: {x0}")
    print(f"  Optimal point: {result.x}")
    print(f"  Objective value: {result.fun:.6f}")
    print(f"  Constraint satisfied: {nlp_constraint(result.x):.2e}")

# Analytical solution
# The constraint is a circle with radius √2
# Minimize x₁ + x₂ on this circle
# Using Lagrange multipliers: ∇f = λ∇h
# [1, 1]^T = λ[2x₁, 2x₂]^T
# This gives x₁ = x₂ and x₁² + x₂² = 2
# So 2x₁² = 2, x₁ = ±1
# Minimum at (-1, -1) with value -2

print("\n" + "="*40)
print("Analytical Solution (Lagrange Multipliers):")
print("="*40)
print("Constraint: Circle with radius √2")
print("Minimize x₁ + x₂ on this circle")
print("Critical points: (1, 1) and (-1, -1)")
print(f"Minimum: f(-1, -1) = -2")
print(f"Maximum: f(1, 1) = 2")
```

PCA

Part 6: PCA with Scikit-learn

While understanding the mathematics is crucial, in practice we use optimized libraries like scikit-learn.

In [ ]:
```
# Load the famous Iris dataset
iris = load_iris()
X_iris = iris.data
y_iris = iris.target
feature_names = iris.feature_names
target_names = iris.target_names

print("Iris Dataset")
print("=" * 60)
print(f"Number of samples: {X_iris.shape[0]}")
print(f"Number of features: {X_iris.shape[1]}")
print(f"Feature names: {feature_names}")
print(f"Target names: {target_names}")
print(f"\nFirst 5 samples:")
print(pd.DataFrame(X_iris[:5], columns=feature_names))
```

In [ ]:
```
# Apply PCA using scikit-learn
# Standardize first
scaler = StandardScaler()
```

```
X_iris_scaled = scaler.fit_transform(X_iris)

# Apply PCA to reduce from 4D to 2D
pca_sklearn = PCA(n_components=2)
X_iris_pca = pca_sklearn.fit_transform(X_iris_scaled)

print("PCA with Scikit-learn")
print("=" * 60)
print(f"Original shape: {X_iris.shape}")
print(f"Transformed shape: {X_iris_pca.shape}")
print(f"\nExplained variance ratio:")
for i, var in enumerate(pca_sklearn.explained_variance_ratio_):
    print(f"  PC{i+1}: {var:.4f} ({var*100:.2f}%)")
print(f"\nTotal variance explained: {np.sum(pca_sklearn.explained_variance_ratio_):

print(f"\nPrincipal components (loadings):")
components_df = pd.DataFrame(
    pca_sklearn.components_,
    columns=feature_names,
    index=['PC1', 'PC2']
)
print(components_df)
```

Part 7: Choosing the Number of Components

Methods for Selecting k

1. **Explained Variance Threshold**: Keep components until you reach a target (e.g., 95% variance)
2. **Scree Plot**: Look for the "elbow" where eigenvalues drop off
3. **Cumulative Variance Plot**: Visualize cumulative variance explained
4. **Cross-Validation**: Use downstream task performance to select k

Rule of Thumb

- **Visualization**: k = 2 or 3
- **Dimensionality Reduction**: Keep 80-95% of variance
- **Noise Reduction**: Keep components with eigenvalues > 1 (Kaiser criterion)

In [ ]:
```
# Fit PCA with all components
pca_full = PCA()
pca_full.fit(X_iris_scaled)

# Print detailed information
print("\nVariance Explained by Each Component:")
print("=" * 60)
for i, (var, cumvar) in enumerate(zip(pca_full.explained_variance_ratio_, cumsum)):
    print(f"PC{i+1}: {var*100:.2f}% (Cumulative: {cumvar:.2f}%)")
```

```
# Recommendation
n_components_90 = np.argmax(cumsum >= 90) + 1
n_components_95 = np.argmax(cumsum >= 95) + 1

print("\n" + "=" * 60)
print("Recommendations:")
print(f"  • For 90% variance: Use {n_components_90} components")
print(f"  • For 95% variance: Use {n_components_95} components")
print(f"  • For visualization: Use 2 components ({cumsum[1]:.1f}% variance)")
```

Part 8: Applications and Use Cases

Common Applications of PCA

1. **Visualization**: Reduce high-dimensional data to 2D/3D for plotting
2. **Noise Reduction**: Remove components with low variance (likely noise)
3. **Feature Extraction**: Create new features that capture most variance
4. **Data Compression**: Reduce storage requirements
5. **Preprocessing**: Improve machine learning model performance
6. **Exploratory Data Analysis**: Understand data structure and relationships

Example: Handwritten Digit Recognition

In [ ]:
```
# Load digits dataset (8x8 images = 64 features)
digits = load_digits()
X_digits = digits.data
y_digits = digits.target

print("Handwritten Digits Dataset")
print("=" * 60)
print(f"Number of samples: {X_digits.shape[0]}")
print(f"Number of features (pixels): {X_digits.shape[1]}")
print(f"Image shape: 8×8 pixels")
print(f"Number of classes: {len(np.unique(y_digits))}")
```

In [ ]:
```
# Apply PCA to digits
scaler_digits = StandardScaler()
X_digits_scaled = scaler_digits.fit_transform(X_digits)

# Reduce from 64D to 2D for visualization
pca_digits = PCA(n_components=2)
X_digits_pca = pca_digits.fit_transform(X_digits_scaled)

print(f"Reduced from {X_digits.shape[1]}D to {X_digits_pca.shape[1]}D")
print(f"Variance explained: {np.sum(pca_digits.explained_variance_ratio_)*100:.2f}%
```

Bootstrap and MoM

In [ ]:
```python
# Import required libraries
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
from scipy.optimize import minimize
import pandas as pd

# Set style for better visualizations
sns.set_style("whitegrid")
plt.rcParams['figure.figsize'] = (12, 6)
plt.rcParams['font.size'] = 11

# Set random seed for reproducibility
np.random.seed(42)

print("Libraries imported successfully!")
```

Part 1: Method of Moments (MoM)

In [ ]:
```python
# Example: Exponential Distribution - Method of Moments

# True parameter
lambda_true = 2.0

# Generate sample data
n = 100
data_exp = np.random.exponential(scale=1/lambda_true, size=n)

# Method of Moments estimator
lambda_mom = 1 / np.mean(data_exp)

print("Exponential Distribution - Method of Moments")
print("=" * 60)
print(f"True parameter: λ = {lambda_true}")
print(f"Sample size: n = {n}")
print(f"Sample mean: {np.mean(data_exp):.4f}")
print(f"\nMoM Estimate: λˆ= {lambda_mom:.4f}")
print(f"Estimation error: {abs(lambda_mom - lambda_true):.4f}")
```

1.3 Example: Normal Distribution

In [ ]:
```python
# Example: Normal Distribution - Method of Moments

# True parameters
mu_true = 50
sigma_true = 10

# Generate sample data
n = 200
data_normal = np.random.normal(loc=mu_true, scale=sigma_true, size=n)

# Method of Moments estimators
```

```
mu_mom = np.mean(data_normal)
sigma2_mom = np.mean((data_normal - mu_mom)**2)  # Biased estimator
sigma_mom = np.sqrt(sigma2_mom)

print("Normal Distribution - Method of Moments")
print("=" * 60)
print(f"True parameters: μ = {mu_true}, σ = {sigma_true}")
print(f"Sample size: n = {n}")
print(f"\nMoM Estimates:")
print(f"  μˆ= {mu_mom:.4f} (error: {abs(mu_mom - mu_true):.4f})")
print(f"  σˆ= {sigma_mom:.4f} (error: {abs(sigma_mom - sigma_true):.4f})")
```

1.4 Example: Bernoulli Distribution

In [ ]:
```
# Example: Bernoulli Distribution - Method of Moments

# True parameter
p_true = 0.7

# Generate sample data
n = 100
data_bernoulli = np.random.binomial(1, p_true, size=n)

# Method of Moments estimator
p_mom = np.mean(data_bernoulli)

print("Bernoulli Distribution - Method of Moments")
print("=" * 60)
print(f"True parameter: p = {p_true}")
print(f"Sample size: n = {n}")
print(f"Number of successes: {np.sum(data_bernoulli)}")
print(f"Number of failures: {n - np.sum(data_bernoulli)}")
print(f"\nMoM Estimate: p̂ = {p_mom:.4f}")
print(f"Estimation error: {abs(p_mom - p_true):.4f}")
```

Part 2: Bootstrap Estimation

2.2 Bootstrap Standard Error

In [ ]:
```
# Example: Bootstrap Standard Error for the Mean

# Original sample
np.random.seed(42)
original_sample = np.random.normal(loc=100, scale=15, size=50)

# Bootstrap parameters
B = 10000  # Number of bootstrap samples
n = len(original_sample)

# Perform bootstrap
bootstrap_means = []
for b in range(B):
    # Resample with replacement
```

```
    bootstrap_sample = np.random.choice(original_sample, size=n, replace=True)
    # Compute statistic (mean)
    bootstrap_means.append(np.mean(bootstrap_sample))

bootstrap_means = np.array(bootstrap_means)

# Bootstrap standard error
se_boot = np.std(bootstrap_means, ddof=1)

# Theoretical standard error (for comparison)
se_theory = np.std(original_sample, ddof=1) / np.sqrt(n)

print("Bootstrap Standard Error Estimation")
print("=" * 60)
print(f"Original sample size: n = {n}")
print(f"Original sample mean: {np.mean(original_sample):.4f}")
print(f"Original sample std: {np.std(original_sample, ddof=1):.4f}")
print(f"Number of bootstrap samples: B = {B}")
print(f"\nBootstrap standard error: {se_boot:.4f}")
print(f"Theoretical standard error: {se_theory:.4f}")
print(f"Difference: {abs(se_boot - se_theory):.4f}")
```

2.3 Bootstrap Confidence Intervals

In [ ]:

```
# Example: Bootstrap Confidence Intervals

# Using the bootstrap means from previous example
alpha = 0.05  # 95% confidence level

# Method 1: Percentile method
ci_percentile = np.percentile(bootstrap_means, [100*alpha/2, 100*(1-alpha/2)])

# Method 2: Normal method
z_critical = stats.norm.ppf(1 - alpha/2)
ci_normal = [np.mean(original_sample) - z_critical * se_boot,
             np.mean(original_sample) + z_critical * se_boot]

# Theoretical CI (for comparison)
ci_theory = stats.t.interval(1-alpha, df=n-1, loc=np.mean(original_sample),
                             scale=se_theory)

print("Bootstrap Confidence Intervals (95%)")
print("=" * 60)
print(f"Point estimate: {np.mean(original_sample):.4f}")
print(f"\n1. Percentile Method:")
print(f"   [{ci_percentile[0]:.4f}, {ci_percentile[1]:.4f}]")
print(f"   Width: {ci_percentile[1] - ci_percentile[0]:.4f}")
print(f"\n2. Normal Method:")
print(f"   [{ci_normal[0]:.4f}, {ci_normal[1]:.4f}]")
print(f"   Width: {ci_normal[1] - ci_normal[0]:.4f}")
print(f"\n3. Theoretical t-interval (for comparison):")
print(f"   [{ci_theory[0]:.4f}, {ci_theory[1]:.4f}]")
print(f"   Width: {ci_theory[1] - ci_theory[0]:.4f}")
```

2.4 Bootstrap for Other Statistics

In [ ]:
```
# Example: Bootstrap for Median

# Generate skewed data (median is more robust than mean for skewed data)
np.random.seed(123)
skewed_data = np.random.exponential(scale=10, size=60)

# Bootstrap for median
B = 10000
n = len(skewed_data)
bootstrap_medians = []

for b in range(B):
    bootstrap_sample = np.random.choice(skewed_data, size=n, replace=True)
    bootstrap_medians.append(np.median(bootstrap_sample))

bootstrap_medians = np.array(bootstrap_medians)

# Bootstrap standard error and CI
se_median = np.std(bootstrap_medians, ddof=1)
ci_median = np.percentile(bootstrap_medians, [2.5, 97.5])

print("Bootstrap for Median (Skewed Data)")
print("=" * 60)
print(f"Sample size: n = {n}")
print(f"Sample mean: {np.mean(skewed_data):.4f}")
print(f"Sample median: {np.median(skewed_data):.4f}")
print(f"\nBootstrap results:")
print(f"  Bootstrap SE(median): {se_median:.4f}")
print(f"  95% CI for median: [{ci_median[0]:.4f}, {ci_median[1]:.4f}]")
```

Example: Bootstrap for Correlation Coefficient

Generate correlated data

np.random.seed(456) n = 50 x = np.random.normal(0, 1, n) y = 0.7 * x + np.random.normal(0, 0.5, n) # Correlation ≈ 0.7

Original correlation

corr_original = np.corrcoef(x, y)[0, 1]

Bootstrap for correlation

B = 10000 bootstrap_corrs = []

for b in range(B): # Resample pairs (important!) indices = np.random.choice(n, size=n, replace=True) x_boot = x[indices] y_boot = y[indices] bootstrap_corrs.append(np.corrcoef(x_boot, y_boot)[0, 1])

bootstrap_corrs = np.array(bootstrap_corrs)

Bootstrap SE and CI

se_corr = np.std(bootstrap_corrs, ddof=1) ci_corr = np.percentile(bootstrap_corrs, [2.5, 97.5])

print("Bootstrap for Correlation Coefficient") print("=" * 60) print(f"Sample size: n = {n}") print(f"Sample correlation: r = {corr_original:.4f}") print(f"\nBootstrap results:") print(f" Bootstrap SE(r): {se_corr:.4f}") print(f" 95% CI for r: [{ci_corr[0]:.4f}, {ci_corr[1]:.4f}]")

2.5 Comparing Bootstrap with Theoretical Results

In [ ]:

```
# Comparison: Bootstrap vs Theory for Sample Mean

# Simulation parameters
true_mu = 50
true_sigma = 10
sample_size = 30
n_simulations = 1000
B = 5000

# Storage
coverage_theory = 0
coverage_bootstrap = 0
widths_theory = []
widths_bootstrap = []

np.random.seed(789)

for sim in range(n_simulations):
    # Generate sample
    sample = np.random.normal(true_mu, true_sigma, sample_size)

    # Theoretical CI
    se_theory = np.std(sample, ddof=1) / np.sqrt(sample_size)
    ci_theory = stats.t.interval(0.95, df=sample_size-1, loc=np.mean(sample), scale
    widths_theory.append(ci_theory[1] - ci_theory[0])
    if ci_theory[0] <= true_mu <= ci_theory[1]:
        coverage_theory += 1

    # Bootstrap CI
    bootstrap_means = []
    for b in range(B):
        boot_sample = np.random.choice(sample, size=sample_size, replace=True)
```

```
        bootstrap_means.append(np.mean(boot_sample))
    ci_boot = np.percentile(bootstrap_means, [2.5, 97.5])
    widths_bootstrap.append(ci_boot[1] - ci_boot[0])
    if ci_boot[0] <= true_mu <= ci_boot[1]:
        coverage_bootstrap += 1

# Results
print("Bootstrap vs Theoretical Comparison")
print("=" * 60)
print(f"True parameters: μ = {true_mu}, σ = {true_sigma}")
print(f"Sample size: n = {sample_size}")
print(f"Number of simulations: {n_simulations}")
print(f"Bootstrap resamples per simulation: B = {B}")
print(f"\nCoverage (should be ≈95%):")
print(f"  Theoretical t-interval: {coverage_theory/n_simulations*100:.2f}%")
print(f"  Bootstrap percentile: {coverage_bootstrap/n_simulations*100:.2f}%")
print(f"\nAverage CI width:")
print(f"  Theoretical: {np.mean(widths_theory):.4f}")
print(f"  Bootstrap: {np.mean(widths_bootstrap):.4f}")
```

K-Means Clustering: A Comprehensive Tutorial

In [ ]:

```
# Import required libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs, load_iris, load_wine
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score, davies_bouldin_score
from scipy.spatial.distance import cdist

# Set style
sns.set_style("whitegrid")
plt.rcParams['figure.figsize'] = (14, 6)
plt.rcParams['font.size'] = 11

# Set random seed
np.random.seed(42)

print("✓ All libraries imported successfully!")
print("✓ Ready to start the K-means tutorial")
```

3.3 Elbow Method Visualization

In [ ]:

```
# Generate data with clear clusters
X_elbow, y_elbow = make_blobs(n_samples=300, centers=4, n_features=2,
                              cluster_std=0.7, random_state=42)
```

```
# Calculate WCSS for different k values
k_range = range(1, 11)
wcss = []
silhouette_scores = []

for k in k_range:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X_elbow)
    wcss.append(kmeans.inertia_)

    # Silhouette score (only for k >= 2)
    if k >= 2:
        silhouette_scores.append(silhouette_score(X_elbow, kmeans.labels_))
    else:
        silhouette_scores.append(0)
```

Part 4: Practical Example - Customer Segmentation

4.1 Problem Setup

Let's apply K-Means to a realistic customer segmentation problem using synthetic data.

In [ ]:
```
# Generate synthetic customer data
np.random.seed(42)
n_customers = 400

# Features: Annual Income (k$), Spending Score (1-100)
# Create 5 distinct customer segments
segment1 = np.random.randn(80, 2) * [5, 8] + [25, 25]   # Low income, low spending
segment2 = np.random.randn(80, 2) * [5, 8] + [25, 75]   # Low income, high spending
segment3 = np.random.randn(80, 2) * [5, 8] + [50, 50]   # Medium income, medium spe
segment4 = np.random.randn(80, 2) * [5, 8] + [75, 25]   # High income, low spending
segment5 = np.random.randn(80, 2) * [5, 8] + [75, 75]   # High income, high spendin

X_customers = np.vstack([segment1, segment2, segment3, segment4, segment5])

# Create DataFrame
df_customers = pd.DataFrame(X_customers, columns=['Annual_Income_k$', 'Spending_Sco

print("Customer Data:")
print("="*60)
print(df_customers.describe())
print(f"\nTotal customers: {len(df_customers)}")
```

In [ ]:
```
# Standardize features
scaler = StandardScaler()
X_customers_scaled = scaler.fit_transform(X_customers)
```

```
# Elbow method
k_range = range(1, 11)
wcss_customers = []
silhouette_customers = []

for k in k_range:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X_customers_scaled)
    wcss_customers.append(kmeans.inertia_)

    if k >= 2:
        silhouette_customers.append(silhouette_score(X_customers_scaled, kmeans.lab
    else:
        silhouette_customers.append(0)
```

In [ ]:
```
# Apply K-Means with k=5
kmeans_final = KMeans(n_clusters=5, random_state=42, n_init=10)
df_customers['Cluster'] = kmeans_final.fit_predict(X_customers_scaled)

# Get centroids in original scale
centroids_scaled = kmeans_final.cluster_centers_
centroids_original = scaler.inverse_transform(centroids_scaled)
```

In [ ]:
```
# Cluster statistics
print("\nCluster Statistics:")
print("="*80)
for i in range(5):
    cluster_data = df_customers[df_customers['Cluster'] == i]
    print(f"\nSegment {i+1}: {segment_names[i]}")
    print(f"  Size: {len(cluster_data)} customers ({len(cluster_data)/len(df_custom
    print(f"  Avg Income: ${cluster_data['Annual_Income_k$'].mean():.1f}k")
    print(f"  Avg Spending Score: {cluster_data['Spending_Score'].mean():.1f}")
    print(f"  Centroid: Income=${centroids_original[i, 0]:.1f}k, Score={centroids_o
```

Dengograms and AgglomerativeClustering

In [ ]:
```
import numpy as np
from scipy.cluster.hierarchy import dendrogram, linkage
from sklearn.cluster import AgglomerativeClustering

# Generate sample data
np.random.seed(42)
X_hier = np.vstack([
    np.random.randn(30, 2) + [2, 2],
    np.random.randn(30, 2) + [8, 2],
    np.random.randn(30, 2) + [5, 8]
])

print(f"Generated {len(X_hier)} points for hierarchical clustering")
```

In [ ]:
```
linkage_matrix = linkage(X_hier, method='ward') # This code generates the Linkage m
# dendrogram(linkage_matrix) # This code is for generating the dendrogram graph
```

8.3 Linkage Methods

In [ ]:
```
linkage_methods = ['single', 'complete', 'average', 'ward']
for idx, method in enumerate(linkage_methods):
    agg = AgglomerativeClustering(n_clusters=3, linkage=method)
    labels = agg.fit_predict(X_hier)
```

8.5 Choosing Number of Clusters

Use silhouette score to find optimal number of clusters.

In [ ]:
```
k_range = range(2, 8)
silhouette_scores_hier = []

for k in k_range:
    agg = AgglomerativeClustering(n_clusters=k, linkage='ward')
    labels = agg.fit_predict(X_hier)
    score = silhouette_score(X_hier, labels)
    silhouette_scores_hier.append(score)

best_k_hier = k_range[np.argmax(silhouette_scores_hier)]
print(f"Optimal k: {best_k_hier}")
```

9. DBSCAN

In [ ]:
```
from sklearn.cluster import DBSCAN
from sklearn.datasets import make_moons, make_circles

# Create datasets where K-Means fails
X_moons, _ = make_moons(n_samples=200, noise=0.05, random_state=42)
X_circles, _ = make_circles(n_samples=200, noise=0.05, factor=0.5, random_state=42)

print("Created challenging datasets for DBSCAN demonstration")
```

9.2 DBSCAN vs K-Means

In [ ]:
```
datasets = [(X_moons, 'Moons'), (X_circles, 'Circles')]

for row, (X, name) in enumerate(datasets):
    # Original data
    axes[row, 0].scatter(X[:, 0], X[:, 1], s=50, alpha=0.6)
    axes[row, 0].set_title(f'{name} Dataset', fontweight='bold')

    # K-Means
    kmeans = KMeans(n_clusters=2, random_state=42, n_init=10)
    labels_km = kmeans.fit_predict(X)
    axes[row, 1].scatter(X[:, 0], X[:, 1], c=labels_km, cmap='viridis', s=50, alpha
    axes[row, 1].set_title(f'K-Means (Fails)', fontweight='bold')
```

```
    # DBSCAN
    dbscan = DBSCAN(eps=0.3, min_samples=5)
    labels_db = dbscan.fit_predict(X)
    axes[row, 2].scatter(X[:, 0], X[:, 1], c=labels_db, cmap='viridis', s=50, alpha
    # Mark noise points
    noise = X[labels_db == -1]
    axes[row, 2].scatter(noise[:, 0], noise[:, 1], c='red', marker='x', s=100, labe
    axes[row, 2].set_title(f'DBSCAN (Succeeds)', fontweight='bold')
    axes[row, 2].legend()

plt.tight_layout()
plt.show()

print("Key Observations:")
print("• K-Means assumes spherical clusters and fails on non-convex shapes")
print("• DBSCAN successfully identifies the true structure")
print("• DBSCAN identifies outliers as noise (red X markers)")
```

9.3 DBSCAN Algorithm Step-by-Step

In [ ]:
```
from sklearn.neighbors import NearestNeighbors

# Create simple dataset
np.random.seed(42)
X_demo = np.random.randn(50, 2)

# Run DBSCAN
dbscan_demo = DBSCAN(eps=0.5, min_samples=5)
labels_demo = dbscan_demo.fit_predict(X_demo)

# Identify point types
core_samples = np.zeros(len(X_demo), dtype=bool)
core_samples[dbscan_demo.core_sample_indices_] = True
noise = labels_demo == -1
border = ~core_samples & ~noise

print(f"Core points: {core_samples.sum()}")
print(f"Border points: {border.sum()}")
print(f"Noise points: {noise.sum()}")
```

9.4 Parameter Selection: K-Distance Graph

In [ ]:
```
# K-distance graph for parameter selection
k = 5  # MinPts
nbrs = NearestNeighbors(n_neighbors=k).fit(X_moons)
distances, indices = nbrs.kneighbors(X_moons)
distances = np.sort(distances[:, k-1], axis=0)
```

9.5 Effect of Parameters

In [ ]:
```
# Try different epsilon values
```

```
eps_values = [0.1, 0.2, 0.3, 0.4]
fig, axes = plt.subplots(1, 4, figsize=(18, 4))

for idx, eps in enumerate(eps_values):
    dbscan = DBSCAN(eps=eps, min_samples=5)
    labels = dbscan.fit_predict(X_moons)

    noise = X_moons[labels == -1]
    n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
    n_noise = list(labels).count(-1)
```

Part 11: Cluster Quality Evaluation

In [ ]:
```
# Evaluate clustering quality on Iris
from sklearn.datasets import load_iris

iris = load_iris()
X_iris = iris.data
y_true = iris.target

# Try different k values
k_values = range(2, 7)
silhouette_scores = []

for k in k_values:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = kmeans.fit_predict(X_iris)
    score = silhouette_score(X_iris, labels)
    silhouette_scores.append(score)
```

11.2 Purity (Requires Ground Truth)

In [ ]:
```
def purity_score(y_true, y_pred):
    # Compute confusion matrix
    contingency_matrix = np.zeros((len(np.unique(y_true)), len(np.unique(y_pred))))
    for i, true_label in enumerate(np.unique(y_true)):
        for j, pred_label in enumerate(np.unique(y_pred)):
            contingency_matrix[i, j] = np.sum((y_true == true_label) & (y_pred == p

    # Purity is sum of max values in each column divided by total
    return np.sum(np.max(contingency_matrix, axis=0)) / np.sum(contingency_matrix)

# Test on Iris
kmeans_iris = KMeans(n_clusters=3, random_state=42, n_init=10)
labels_iris = kmeans_iris.fit_predict(X_iris)

purity = purity_score(y_true, labels_iris)
```

11.3 Entropy (Requires Ground Truth)

In [ ]:
```
def entropy_score(y_true, y_pred):
    from scipy.stats import entropy as scipy_entropy

    total_entropy = 0
    n_samples = len(y_true)

    for cluster_label in np.unique(y_pred):
        cluster_mask = y_pred == cluster_label
        cluster_size = np.sum(cluster_mask)

        if cluster_size == 0:
            continue

        # Distribution of true labels in this cluster
        label_counts = np.bincount(y_true[cluster_mask], minlength=len(np.unique(y_
        label_probs = label_counts / cluster_size

        # Entropy of this cluster
        cluster_entropy = scipy_entropy(label_probs, base=2)

        # Weighted by cluster size
        total_entropy += (cluster_size / n_samples) * cluster_entropy

    return total_entropy

entropy = entropy_score(y_true, labels_iris)
print(f"Entropy Score: {entropy:.3f}")
print(f"Purity Score: {purity:.3f}")
print("\nLower entropy = better clustering")
print("Higher purity = better clustering")
```

11.4 RAND Index (Requires Ground Truth)

In [ ]:
```
from sklearn.metrics import rand_score, adjusted_rand_score

rand = rand_score(y_true, labels_iris)
adj_rand = adjusted_rand_score(y_true, labels_iris)

print(f"RAND Index: {rand:.3f}")
print(f"Adjusted RAND Index: {adj_rand:.3f}")
print("\nAdjusted RAND Index:")
print("• Range: [-1, 1]")
print("• 1: Perfect match")
print("• 0: Random labeling")
print("• Negative: Worse than random")
```

12. Foundations of Machine Learning: Regression

2. Simple Linear Regression

In [ ]:
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

# Generate sample data for simple linear regression
np.random.seed(42)
X = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10]).reshape(-1, 1)
y = 2 + 1.5 * X.flatten() + np.random.normal(0, 1, 10)

# Create and fit the model
model = LinearRegression()
model.fit(X, y)

# Get predictions
y_pred = model.predict(X)
```

3. Multiple Linear Regression

In [ ]:
```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Sample data with two independent variables
np.random.seed(42)
X_multi = np.array([
    [1, 1], [1, 2], [2, 2], [2, 3], [3, 3],
    [3, 4], [4, 4], [4, 5], [5, 5], [5, 6]
])
y_multi = 1 + 2*X_multi[:, 0] + 1.5*X_multi[:, 1] + np.random.normal(0, 0.5, 10)

# Create and fit the model
model_multi = LinearRegression()
model_multi.fit(X_multi, y_multi)

# Get predictions
y_pred_multi = model_multi.predict(X_multi)

# Calculate metrics
mse = mean_squared_error(y_multi, y_pred_multi)
r2 = r2_score(y_multi, y_pred_multi)

print(f'Intercept (w₀): {model_multi.intercept_:.4f}')
print(f'Coefficients: w₁ = {model_multi.coef_[0]:.4f}, w₂ = {model_multi.coef_[1]:.
print(f'Mean Squared Error: {mse:.4f}')
print(f'R² Score: {r2:.4f}')
```

4. Ordinary Least Squares (OLS)

In [ ]:
```
# Implement OLS manually using the normal equation
import numpy as np
from numpy.linalg import inv
```

```python
# Generate sample data
np.random.seed(42)
n_samples = 20
X = np.linspace(0, 10, n_samples).reshape(-1, 1)
y = 2 + 1.5 * X.flatten() + np.random.normal(0, 1.5, n_samples)

# Add bias term (column of ones)
X_with_bias = np.hstack([np.ones((n_samples, 1)), X])

# OLS solution using normal equation
# w_hat = (X^T X)^-1 X^T y
XTX = X_with_bias.T @ X_with_bias
XTy = X_with_bias.T @ y
w_hat = inv(XTX) @ XTy

print("OLS Solution using Normal Equation:")
print(f"Intercept (w₀): {w_hat[0]:.4f}")
print(f"Slope (w₁): {w_hat[1]:.4f}")

# Calculate predictions and residuals
y_pred = X_with_bias @ w_hat
residuals = y - y_pred
ssr = np.sum(residuals**2)

print(f"\nSum of Squared Residuals: {ssr:.4f}")
print(f"Mean Squared Error: {ssr/n_samples:.4f}")

# Compare with scikit-learn
from sklearn.linear_model import LinearRegression
model_sklearn = LinearRegression()
model_sklearn.fit(X, y)

print(f"\nComparison with scikit-learn:")
print(f"Intercept (sklearn): {model_sklearn.intercept_:.4f}")
print(f"Slope (sklearn): {model_sklearn.coef_[0]:.4f}")
```

In [ ]:

```python
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

# Demonstrate OLS as projection for a simple 2D case
np.random.seed(42)

# Create sample data
n_samples = 15
X = np.linspace(0, 10, n_samples).reshape(-1, 1)
y = 2 + 1.5 * X.flatten() + np.random.normal(0, 1.5, n_samples)

# Add bias term
X_with_bias = np.hstack([np.ones((n_samples, 1)), X])

# OLS solution
from numpy.linalg import inv
w_hat = inv(X_with_bias.T @ X_with_bias) @ X_with_bias.T @ y
y_pred = X_with_bias @ w_hat
```

```
residuals = y - y_pred

print("Linear Algebraic Interpretation of OLS:")
print("=" * 50)
print(f"\nDesign Matrix X* shape: {X_with_bias.shape}")
print(f"Observation vector y shape: {y.shape}")
print(f"Predicted vector ŷ shape: {y_pred.shape}")
print(f"Residual vector e shape: {residuals.shape}")

# Verify orthogonality: X*^T e should be approximately zero
orthogonality_check = X_with_bias.T @ residuals
print(f"\nOrthogonality Check (X*^T e):")
print(f"  {orthogonality_check}")
print(f"  (Should be close to zero, max value: {np.max(np.abs(orthogonality_check))

# Calculate projection matrix
P = X_with_bias @ inv(X_with_bias.T @ X_with_bias) @ X_with_bias.T
print(f"\nProjection Matrix P shape: {P.shape}")
print(f"Predicted values ŷ = P y")
print(f"Verification: ||P y - ŷ|| = {np.linalg.norm(P @ y - y_pred):.2e}")
```

6. Total Least Squares (TLS)

In [ ]:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.linalg import svd

# Generate sample data with errors in both X and y
np.random.seed(42)
n_samples = 20

# True relationship: y = 2 + 1.5*x
x_true = np.linspace(0, 10, n_samples)
y_true = 2 + 1.5 * x_true

# Add errors to both X and y
X_noisy = x_true + np.random.normal(0, 0.5, n_samples)
y_noisy = y_true + np.random.normal(0, 0.8, n_samples)

# Prepare data matrices
X_matrix = X_noisy.reshape(-1, 1)
X_with_bias = np.hstack([np.ones((n_samples, 1)), X_matrix])

# Method 1: OLS solution
from numpy.linalg import inv
w_ols = inv(X_with_bias.T @ X_with_bias) @ X_with_bias.T @ y_noisy
y_pred_ols = X_with_bias @ w_ols

# Method 2: TLS solution using SVD
# Augment the matrix [X | y]
A_augmented = np.hstack([X_with_bias, y_noisy.reshape(-1, 1)])

# Perform SVD
U, s, Vt = svd(A_augmented, full_matrices=True)
```

```python
# The TLS solution is related to the right singular vectors
# corresponding to the smallest singular value
V = Vt.T
v_last = V[:, -1]  # Last column (corresponds to smallest singular value)

# Extract TLS coefficients
# If v_last = [v1, v2, v3], then w_tls = -[v1, v2] / v3
w_tls = -v_last[:-1] / v_last[-1]
y_pred_tls = X_with_bias @ w_tls

print("Total Least Squares (TLS) vs OLS Comparison:")
print("=" * 50)
print(f"\nOLS Solution:")
print(f"  w₀ (intercept) = {w_ols[0]:.4f}")
print(f"  w₁ (slope) = {w_ols[1]:.4f}")

print(f"\nTLS Solution:")
print(f"  w₀ (intercept) = {w_tls[0]:.4f}")
print(f"  w₁ (slope) = {w_tls[1]:.4f}")

print(f"\nTrue Parameters:")
print(f"  w₀ (intercept) = 2.0000")
print(f"  w₁ (slope) = 1.5000")

# Calculate residuals
residuals_ols = y_noisy - y_pred_ols
residuals_tls = y_noisy - y_pred_tls

# For TLS, we also calculate the perpendicular distances
# Perpendicular distance from point (x_i, y_i) to line ax + by + c = 0
# is |ax_i + by_i + c| / sqrt(a^2 + b^2)
# Our line is: w_1*x - y + w_0 = 0
a_tls, b_tls, c_tls = w_tls[1], -1, w_tls[0]
perp_distances_tls = np.abs(a_tls * X_noisy + b_tls * y_noisy + c_tls) / np.sqrt(a_

print(f"\nError Metrics:")
print(f"  OLS - Sum of Squared Residuals: {np.sum(residuals_ols**2):.4f}")
print(f"  TLS - Sum of Squared Residuals: {np.sum(residuals_tls**2):.4f}")
print(f"  TLS - Sum of Squared Perpendicular Distances: {np.sum(perp_distances_tls*
```